KARTGEN HIREFLOW AI

Complete Product, UX, Data & Engineering Source of Truth

Version 2 • Consolidated from the complete product discussion • 18 October 2026

One document: every confirmed decision, scenario, open question, implementation plan, database field, API contract, bottleneck, risk, improvement, diagram and acceptance test. This document is a proposed architecture, not a claim that software has already been built or tested.

How to read this document

- PART A: Product vision, business case, scope and UX screens.
- PART B: Every ingestion, identity, resume-version, application and JD matching rule.
- PART C: Physical PostgreSQL data dictionary, relationships, constraints and indexes.
- PART D: React, Spring Boot, APIs, queues, workers, deployment and security.
- PART E: Bottleneck register, mitigations, test plan, phased implementation and improvements.
- Every recommendation is marked as recommended rather than silently treated as approved.

A1. Original problem and the product promise

HR receives high volumes of applications by email. Resumes are missed, scattered, duplicated and manually copied into spreadsheets. Hiring teams need one clean workspace that captures candidate submissions automatically, extracts structured details, allows JD-first search and makes original resumes easy to view.

• Primary UI: minimal off-white background and lavender #CBC3E3 accents, responsive React dashboard.

• Original dashboard sketch: left sidebar, top search, All Resumes, Java, React, Full Stack and dropdown filters, large candidate results area, logout.

• HR can enter or upload a job description; candidate list filters immediately from stored structured profiles, not by reparsing every PDF.

• Required fields include name, email, phone, LinkedIn, technology stack, experience, location, education, resume and application source.

• Automate incoming direct candidate mail AND forwarded mail from HR, agencies, recruiters and placement colleges.

• Living Candidate Profile: one candidate identity per tenant; newest validated resume for current matching and viewing; preserve historical application evidence.

• System must remain responsive during ingestion, OCR, AI, matching and resume preview.

A2. Product principles, positives and negatives

Positive / value	Negative / risk	Design response
Automatically captures applications	Email access trust and privacy	Dedicated inbox / controlled forwarding default
Reusable candidate talent pool	Existing ATS competitors	Focus on easy onboarding and inbox-to-search
Current candidate skills and experience	AI may invent or misread data	Evidence, validation, human review
Fast JD matching	Stale or unfair relevance rankings	Versioned explainable scoring, manual override
Lower duplicate storage / AI cost	Identity false merges	Conservative evidence-based matching
Commercial multi-tenant SaaS	Cross-tenant data exposure	Tenant authorization + constraints + tests

A3. Scope, roles and permissions

Phase	Included	Not included by default
MVP	Email intake, upload, extraction, dedup, Living Profile, JD filters, viewer, shortlist, audit, security	Full payroll, onboarding, HRMS, automated rejection
Phase 2	Agency workflows, richer matching, exports, reporting, ATS integrations	Dedicated OpenSearch until needed
Enterprise	Customer cloud, SSO/SCIM, advanced governance, assessments	Certification claims before audits

Role	Permissions
Org Owner	Manage tenant, billing, security, retention, members, integrations
HR Admin	Connect authorized sources, create jobs, review all permitted candidates, manage pipeline
Recruiter	Review assigned jobs/candidates, shortlist, notes, interview stages
Hiring Manager	Read assigned candidate/job data and submit evaluations
Auditor	Read audit and processing history with restricted PII
Platform Operator	Infrastructure only by default; exceptional support access is time-bound and audited

A4. Screen-by-screen minimal UI specification

Login / invite

SSO or secure login, organization selection, MFA for privileged roles; no unnecessary fields.

Email setup

Three options: dedicated recruitment inbox, controlled forwarding, optional broad OAuth with explicit permission disclosure.

Dashboard

Sidebar: Dashboard, Candidates, Jobs, Review Queue, Inbox Health, Settings. Top: global search, JD selector, tech chips, advanced filters.

Candidate list

25-row server-side page; name, current title, validated experience, skills, location, updated date, source, match score, stage, View Resume.

JD creation

Paste text or upload JD; review extracted requirements; version and save; background match progress.

Candidate profile

Living Profile summary, current resume, versions, applications, evidence, verification flags, notes and hiring actions.

Resume viewer

Right drawer or dedicated page; show metadata instantly; fetch selected PDF on demand; historical version selector.

Review queue

Uncertain identity, unreadable attachment, mismatched experience, unsupported file, stale resume, failed parsing.

Pipeline

Job-specific New, Reviewed, Shortlisted, Interview, Offer, Hired, Rejected with history.

Security settings

Source connection, exact OAuth scope, revoke, processing logs, export, deletion and retention controls.

Operations (admin)

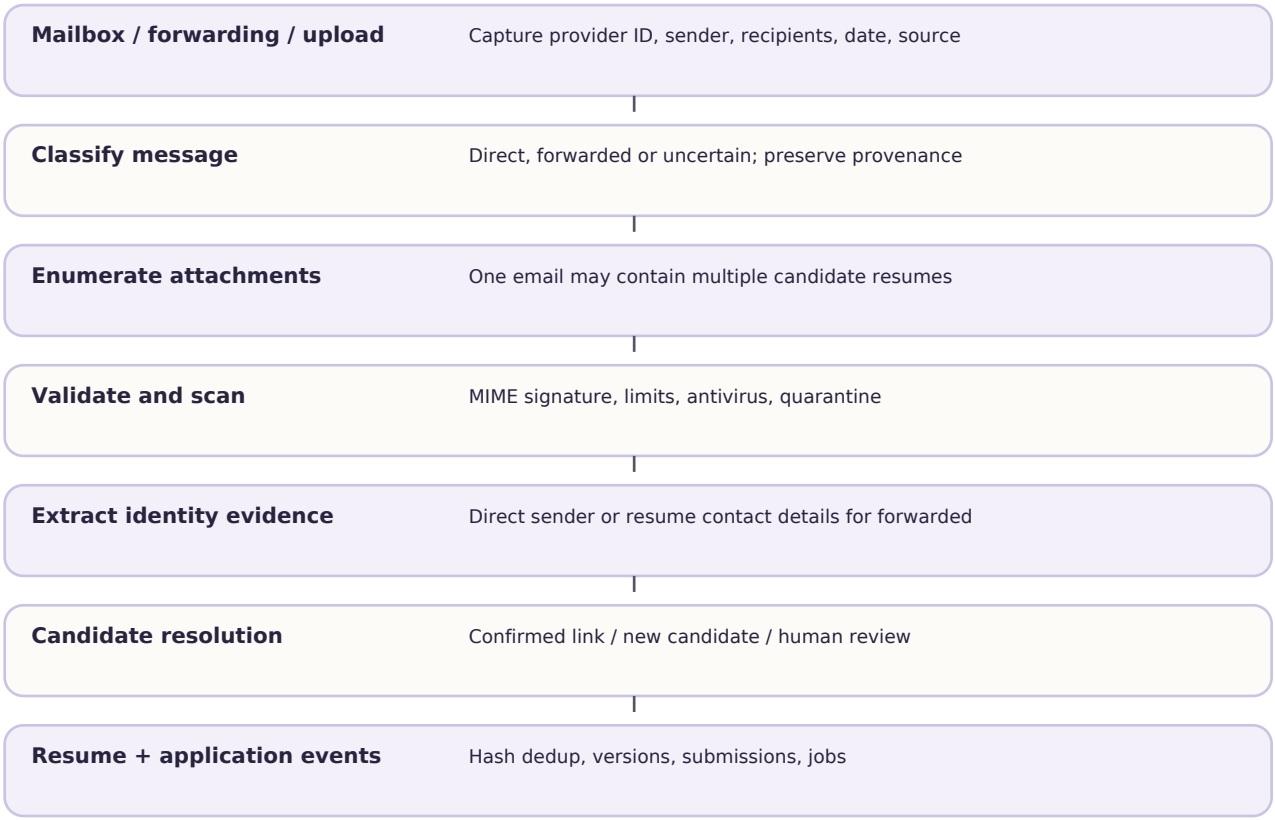
Sync lag, queue age, worker throughput, OCR errors, AI cost, failed jobs, p95 latency.

Candidate list interaction

Search or choose JD	Read saved JD requirements; no resume reparse
Filter candidates	Indexed tenant-scoped query + cursor/page
Select candidate	Load structured profile and evidence
Click View Resume	Authorize then issue short-lived signed S3 URL
Take hiring action	Optimistic version check + stage history

B1. Email intake: direct versus forwarded

Dual intake, shared processing pipeline



Case	Candidate key	Provenance
Direct candidate email	Sender is strong signal; cross-check resume	Store sender, provider message ID
Forwarded HR/agency/college	Resume email + corroborating evidence; NEVER forwarder as candidate	Store actual forwarder and claimed original sender
10 attachments in one email	Evaluate each attachment separately	One inbound message, many attachment/submission records
No email on resume	Phone/LinkedIn/name as hints; review if uncertain	Preserve attachment and source
Changed candidate email	Candidate identifiers table and evidence	Old/new emails remain associated when confirmed

B2. Mail privacy, trust and onboarding

• Preferred: connect a dedicated recruitment-only mailbox. Alternative: company-controlled forwarding to unique tenant intake address, without source mailbox OAuth.

• Optional existing Gmail/Outlook OAuth: disclose that read permissions may expose more than filtered recruitment messages; filtering inside app is not a permission boundary.

• Store OAuth tokens encrypted with KMS; refresh safely; rotate; revoke and disconnect promptly; no mailbox passwords.

• Do not promise zero risk: forwarding rules may accidentally send unrelated mail; implement dry-run, sampling and privacy incident handling.

• Restrict AI input to necessary resume text; minimize retained email body; prohibit unauthorized training or cross-customer reuse.

• Publish security architecture, subprocessors, retention/deletion policies, DPA, independent tests when obtained, and export workflow.

• Google restricted Gmail scopes may require verification/security assessment depending on product and data handling; verify current provider requirements before launch.

• Enterprise Microsoft 365: evaluate mailbox-scoped application permissions under Exchange controls; validate effective permissions.

B3. Mail sync and failure recovery

Incremental sync with reconciliation

Provider notification / polling	Treat notifications as hints, not guaranteed delivery
Load mailbox checkpoint	Provider history ID / delta token per connection
Fetch changed messages	Bounded concurrency, rate limit, backoff
Upsert message registry	Unique tenant + connection + provider ID
Create durable processing work	Transactional outbox then queue
Advance checkpoint safely	Only after durable message recording
Periodic reconciliation	Backfill gaps, handle invalid cursors
Ops alert	Lag, revoked token, repeated failure

B4. Parsing, OCR, AI and versioning

Async resume processing

Receive file	S3 quarantine, size/type validation, SHA-256
File hash lookup	Same tenant + same bytes: reuse file and extraction
Malware scan and sandbox	Reject/quarantine dangerous content
PDFBox/Tika DOCX/PDF	Extract text with layout metadata
OCR fallback	Only for unreadable/scanned pages; bounded pages
AI structured extraction	Strict JSON schema, prompt injection isolation
Validate / normalize	Email, phone, dates, skills, provenance
Resolve identity	Link / create / review; do not false merge
Promote current version	Newest reliable document by evidence/order
Recompute affected matches	Async, version-keyed; never block dashboard

• No automatic experience increment just because a year elapsed. Infer experience only from supported employment periods or reliable updated resume evidence.

• A resume arriving later can be older content forwarded by an agency. Receipt time alone must not replace a newer validated profile.

• If V2 finishes before V1, V1 must never overwrite V2. Use monotonic sequence/version and guarded transaction.

• Current resume is distinct from the resume submitted for a historical application.

• Same file with a different filename has same SHA-256; identical hash is file dedup, not person identity proof.

• No usable text, unsupported files and uncertain identity become visible review cases.

B5. All reapplication edge cases

Scenario	Candidate	Resume / parsing	Application / submission
Same person same job same PDF 10:00 + 10:15	One identity	One file; one extraction	Preserve two source events; one active row as policy
Same person same job changed PDF 11:30	One identity	New version; parse once	Update existing active application; history intact
Same person different job same PDF	One identity	Reuse file	New job application
Same person next year 5+ to 6+	One identity	Validate new resume and experience	New application attempt; historical decision preserved
Same email but conflicting identity	No automatic merge	Quarantine ambiguous association	Review queue
Agency forwards old PDF after new PDF	One identity	Do not downgrade current version	Keep submission provenance
Provider redelivers identical message	One identity	No new work	Idempotency suppresses duplicate event
Candidate re-applies after rejection	One identity	Use latest validated profile	New attempt, do not auto-reject

B6. JD filtering and matching rules

• Analyze JD once per immutable version: title, must-have and preferred skills, experience range, location, employment mode and domain constraints.

• Retrieve candidates with tenant-scoped normalized skills and indexes; do not invoke OCR or LLM on every JD search.

• Show indexed initial results quickly; optionally compute expensive relevance in background and persist keyed by (job_version, candidate_profile_version, model_version).

• Score explainably: supported skills, years of relevant work, recency, location/work preferences, evidence confidence; do not treat missing fields as false facts.

• Keep human hiring decisions in control. Do not automatically reject candidates solely on AI score or inferred demographic attributes.

• Profile promotion invalidates affected matches and triggers background recalculation; keep old score visible as stale until refreshed if needed.

C1. PostgreSQL relationship overview

Logical relational data model

organizations -> organization_users	Tenant membership, roles and policies
mail_connections -> inbound_messages	One connection receives many messages
inbound_messages -> inbound_attachments	One message may contain many candidate files
candidates -> candidate_identifiers	One identity, multiple emails/phones/LinkedIn IDs
candidates -> resume_versions	Unique candidate-linked resume evidence
resume_files -> resume_versions	Deduplicated physical object referenced by versions
jobs -> job_versions -> job_matches	Immutable JD requirements and cached scores
applications -> application_submissions	Job application/attempt versus distinct events
candidates -> profile_versions	Current profile plus history, corrections
audit_events + outbox_events	Traceability and durable asynchronous changes

C2. Full physical database data dictionary

The following is the proposed physical schema. All tenant-owned tables must use organization_id, tenant-consistent foreign keys and authorization checks. Types and column constraints are baseline designs to finalize in Flyway migrations. Rows below are field definitions, not sample candidate data.

organizations (7 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Tenant ID
name	varchar(200) NOT NULL	Organization name
slug	varchar(100) UNIQUE	Tenant routing
status	varchar(24)	ACTIVE/SUSPENDED
default_timezone	varchar(64)	Display timestamps
retention_policy_id	uuid nullable FK	Data retention
created_at	timestamptz	Created

users (6 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	User
email	varchar(320) unique	Login identity
display_name	varchar(200)	Name
auth_subject	varchar(255) unique	OIDC subject
mfa_enabled	boolean	Security
created_at	timestamptz	Created

organization_users (6 columns)

Column	PostgreSQL type / rule	Purpose
organization_id	uuid FK	Tenant
user_id	uuid FK	Member
role	varchar(32)	OWNER/ADMIN/RECRUITER/MANAGER/AUDITOR
status	varchar(24)	INVITED/ACTIVE/REVOKED
joined_at	timestamptz	Joined
version	bigint	Optimistic locking

mail_connections (11 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Connection
organization_id	uuid FK	Tenant
provider	varchar(30)	GMAIL/MS_GRAPH/FORWARDING
mailbox_address	varchar(320)	Authorized address
oauth_secret_ref	text nullable	Secret reference not token
permission_scope	text	Actual scopes
sync_cursor	text nullable	History/delta checkpoint
last_success_at	timestamptz nullable	Health
status	varchar(30)	ACTIVE/REAUTH/ERROR
created_by	uuid FK	Actor
created_at	timestamptz	Created

inbound_messages (14 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Message
organization_id	uuid FK	Tenant
connection_id	uuid FK	Source
provider_message_id	varchar(500)	Provider idempotency key
internet_message_id	varchar(500) nullable	Header evidence
sender_email	varchar(320)	Actual sender
sender_name	varchar(200) nullable	Sender display
original_sender_claim	varchar(320) nullable	Forwarded claimed sender
source_type	varchar(30)	DIRECT/FORWARDED/UNCERTAIN
subject	text nullable	Minimized subject
received_at	timestamptz	Received
processing_status	varchar(30)	State
failure_code	varchar(80) nullable	Error
created_at	timestamptz	Recorded

inbound_attachments (12 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Attachment
organization_id	uuid FK	Tenant
message_id	uuid FK	Parent
provider_attachment_id	varchar(500) nullable	Source key
attachment_index	int	Stable ordinal
original_filename	text	Display name
declared_mime	varchar(120)	Untrusted MIME
detected_mime	varchar(120)	Validated MIME
size_bytes	bigint	Limits
sha256	char(64)	Content hash
resume_file_id	uuid nullable FK	Stored object
status	varchar(30)	Scan/parse state

resume_files (10 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Physical object
organization_id	uuid FK	Tenant
sha256	char(64)	Dedup bytes
s3_bucket	varchar(200)	Private bucket
s3_object_key	text	Encrypted object key
size_bytes	bigint	Size
mime_type	varchar(120)	Validated type
malware_status	varchar(30)	PENDING/CLEAN/QUARANTINED
storage_class	varchar(40)	Lifecycle
created_at	timestamptz	Stored

candidates (15 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Identity
organization_id	uuid FK	Tenant
display_name	varchar(250) nullable	Current name
primary_email	varchar(320) nullable	Convenience field
primary_phone	varchar(50) nullable	Current contact
linkedin_url	text nullable	Profile link
current_resume_version_id	uuid nullable FK	Current validated version
current_profile_version_id	uuid nullable FK	Current snapshot
current_experience_months	int nullable	Evidence-based
experience_confidence	varchar(24)	VERIFIED/ESTIMATED/UNKNOWN
current_city	varchar(150) nullable	Location
last_application_at	timestamptz nullable	Recent activity
profile_updated_at	timestamptz	Last update
status	varchar(24)	ACTIVE/MERGED/ARCHIVED
version	bigint	Optimistic lock

candidate_identifiers (9 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Identifier
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Owner
identifier_type	varchar(25)	EMAIL/PHONE/LINKEDIN
normalized_value	varchar(500)	Search key
source_type	varchar(30)	RESUME/SENDER/HR
verification_status	varchar(24)	UNVERIFIED/VERIFIED/CONFLICT
first_seen_at	timestamptz	First evidence
last_seen_at	timestamptz	Last evidence

resume_versions (15 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Logical version
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Candidate
resume_file_id	uuid FK	Physical file
version_number	int	Candidate sequence
received_at	timestamptz	Arrival
document_date	date nullable	Explicit document date
source_attachment_id	uuid nullable FK	Provenance
parse_status	varchar(30)	PENDING/READY/FAILED
validation_status	varchar(30)	VALID/REVIEW/INVALID
parser_version	varchar(50)	Reproducibility
extracted_text_ref	text nullable	Private extracted text
processed_at	timestamptz nullable	Completed
promoted_at	timestamptz nullable	Current promotion
superseded_at	timestamptz nullable	Replaced

candidate_profile_versions (14 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Snapshot
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Candidate
resume_version_id	uuid FK	Evidence
profile_version	bigint	Monotonic
full_name	varchar(250) nullable	Parsed name
headline	varchar(250) nullable	Role
experience_months	int nullable	Supported duration
experience_basis	varchar(50)	RESUME_DATES/CLAIM/HR
location_text	varchar(250) nullable	Location
education_json	jsonb	Education evidence
source_evidence_json	jsonb	Field provenance
verified_by	uuid nullable FK	Reviewer
created_at	timestamptz	Snapshot

candidate_experience_snapshots (10 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Snapshot
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Person
resume_version_id	uuid FK	Source
total_months	int nullable	Experience
approximate	boolean	Claim precision
source_as_of_date	date nullable	Evidence date
calculation_method	varchar(50)	Claim or employment periods
confidence	varchar(24)	Confidence
created_at	timestamptz	Stored

skills (4 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Skill
canonical_name	varchar(150) unique	Spring Boot
category	varchar(80)	Backend/Cloud
active	boolean	Catalog

skill_aliases (3 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Alias
skill_id	uuid FK	Canonical skill
alias_normalized	varchar(150) unique	springboot

candidate_skills (7 columns)

Column	PostgreSQL type / rule	Purpose
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Candidate
skill_id	uuid FK	Skill
profile_version_id	uuid FK	Evidence snapshot
evidence_type	varchar(30)	EXPLICIT/INFERRED/HR
confidence	numeric(4,3)	0..1
last_confirmed_at	timestamptz	Freshness

jobs (9 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Job
organization_id	uuid FK	Tenant
title	varchar(250)	Role
department	varchar(150) nullable	Department
location	varchar(200) nullable	City
status	varchar(24)	DRAFT/OPEN/CLOSED
current_version_id	uuid nullable FK	Current JD
created_by	uuid FK	Creator
created_at	timestamptz	Created

job_versions (11 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	JD version
organization_id	uuid FK	Tenant
job_id	uuid FK	Job
version_number	int	Sequence
jd_text_ref	text nullable	Private JD content
jd_hash	char(64)	Dedup version
min_experience_months	int nullable	Experience filter
max_experience_months	int nullable	Experience filter
requirements_json	jsonb	Validated structured JD
parser_version	varchar(50)	Extraction version
created_at	timestamptz	Created

job_skill_requirements (5 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Requirement
job_version_id	uuid FK	JD
skill_id	uuid FK	Canonical skill
priority	varchar(24)	MUST/PREFERRED
weight	numeric(6,3)	Ranking weight

job_matches (10 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Match
organization_id	uuid FK	Tenant
job_version_id	uuid FK	JD version
candidate_id	uuid FK	Candidate
profile_version_id	uuid FK	Candidate snapshot
scorer_version	varchar(50)	Model version
score	numeric(6,3)	0..100
explanation_json	jsonb	Evidence breakdown
status	varchar(24)	READY/STALE/ERROR
computed_at	timestamptz	Calculated

applications (11 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Application
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Person
job_id	uuid nullable FK	May be unassigned
attempt_number	int	Reapplication attempt
current_stage	varchar(32)	NEW/SHORTLISTED/etc
applied_at	timestamptz	First event
last_submitted_at	timestamptz	Latest event
submitted_resume_version_id	uuid nullable FK	Historical evidence
assigned_recruiter_id	uuid nullable FK	Owner
version	bigint	Optimistic lock

application_submissions (11 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Event
organization_id	uuid FK	Tenant
application_id	uuid FK	Application
inbound_message_id	uuid nullable FK	Source email
attachment_id	uuid nullable FK	One of many attachments
resume_version_id	uuid nullable FK	Submitted resume
source_type	varchar(30)	DIRECT/FORWARDED/UPLOAD
submission_fingerprint	char(64)	Event idempotency
submitted_at	timestamptz	Timestamp
event_class	varchar(32)	NEW/REPEAT/UPDATE
created_at	timestamptz	Stored

application_stage_history (8 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	History
organization_id	uuid FK	Tenant
application_id	uuid FK	Application
from_stage	varchar(32) nullable	Previous
to_stage	varchar(32)	New stage
changed_by	uuid FK	Actor
reason	text nullable	Audit note
changed_at	timestamptz	When

candidate_notes (7 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Note
organization_id	uuid FK	Tenant
candidate_id	uuid FK	Person
application_id	uuid nullable FK	Context
author_id	uuid FK	Author
body	text	Note
created_at	timestamptz	Created

identity_review_cases (9 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Review
organization_id	uuid FK	Tenant
candidate_id	uuid nullable FK	Candidate
attachment_id	uuid nullable FK	Source
candidate_matches_json	jsonb	Possible identities
reason_code	varchar(50)	CONFLICT/LOW_CONFIDENCE
status	varchar(24)	OPEN/RESOLVED
resolved_by	uuid nullable FK	Reviewer
resolved_at	timestamptz nullable	Resolution

processing_jobs (10 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Job
organization_id	uuid FK	Tenant
job_type	varchar(40)	OCR/PARSE/MATCH
aggregate_id	uuid	Target
idempotency_key	varchar(250)	Unique work key
status	varchar(24)	PENDING/RUNNING/etc
attempt_count	int	Retry count
next_attempt_at	timestamptz nullable	Backoff
error_code	varchar(100) nullable	Failure
created_at	timestamptz	Queued

outbox_events (7 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Outbox
organization_id	uuid FK	Tenant
event_type	varchar(80)	Event name
aggregate_id	uuid	Aggregate
payload_json	jsonb	Minimal event payload
published_at	timestamptz nullable	Delivery
created_at	timestamptz	Committed

audit_events (8 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Audit
organization_id	uuid FK	Tenant
actor_id	uuid nullable FK	User/system
action	varchar(100)	VIEW_RESUME/etc
entity_type	varchar(50)	Target type
entity_id	uuid nullable	Target
metadata_json	jsonb	Redacted details
occurred_at	timestamptz	When

retention_policies (7 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Policy
organization_id	uuid FK	Tenant
resume_retention_days	int	Configured retention
message_retention_days	int	Raw mail retention
audit_retention_days	int	Audit policy
deletion_grace_days	int	Grace
updated_at	timestamptz	Updated

data_deletion_requests (7 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Request
organization_id	uuid FK	Tenant
candidate_id	uuid nullable FK	Target
requested_by	uuid FK	Requester
request_type	varchar(30)	DELETE/EXPORT
status	varchar(24)	PENDING/DONE
completed_at	timestamptz nullable	Completion

ai_usage_events (9 columns)

Column	PostgreSQL type / rule	Purpose
id	uuid PK	Usage
organization_id	uuid FK	Tenant
processing_job_id	uuid FK	Job
provider	varchar(80)	Provider
model_name	varchar(100)	Model
input_tokens	int	Input
output_tokens	int	Output
estimated_cost	numeric(12,6)	Budget
created_at	timestamptz	When

C3. Essential indexes, uniqueness and integrity

Table	Recommended index / constraint	Why
inbound_messages	UNIQUE (organization_id, connection_id, provider_message_id)	Retry-safe ingestion
inbound_attachments	UNIQUE (message_id, attachment_index)	Multi-attachment identity
resume_files	UNIQUE (organization_id, sha256)	One physical file per unique tenant content
resume_versions	UNIQUE (organization_id, candidate_id, version_number)	Stable version history
candidate_identifiers	INDEX (organization_id, identifier_type, normalized_value)	Identity lookup; verified uniqueness policy separately
candidates	INDEX (organization_id, last_application_at DESC, id)	Recent dashboard
candidate_skills	INDEX (organization_id, skill_id, candidate_id)	Technology filtering
job_matches	UNIQUE (job_version_id, candidate_id, profile_version_id, scorer_version)	Reusable score
applications	INDEX (organization_id, job_id, current_stage, last_submitted_at DESC)	Job pipeline
application_submissions	UNIQUE (organization_id, submission_fingerprint)	Distinct events without duplicate retries
processing_jobs	UNIQUE (organization_id, idempotency_key)	Idempotent background work
outbox_events	INDEX (published_at, created_at)	Reliable publisher

• Use composite (organization_id,id) referenced keys to enforce same-tenant foreign keys where feasible; validate cross-table current pointers.

• Do NOT create UNIQUE(candidate_id,job_id): legitimate repeat applications must remain possible.

• Use a guarded transaction to promote current_resume_version_id; older workers cannot overwrite newer validated profiles.

• Normalize emails conservatively (trim, lowercase domain; avoid unsafe Gmail dot/plus stripping for arbitrary providers).

• For identity identifiers, enforce verified ownership conflict policy; shared emails must not force false merges.

• Avoid deleting physical resume objects while referenced by current profile, retained historical submissions or legal holds.

C4. Candidate version promotion algorithm

Atomic profile promotion

Candidate resolution confirmed	Tenant-scoped candidate lock/version
Parse new resume	Capture immutable extraction + source evidence
Compare freshness and validity	Content dates, receipt sequence, verified source
If ambiguous	Create review case; keep existing current pointer
If eligible	Insert profile snapshot and skill evidence
Guarded UPDATE	Only promote if newer approved precedence
Publish outbox event	CandidateProfileUpdated(profileVersion)
Async match refresh	Invalidate/recalculate affected job matches

D1. Spring Boot modular monolith

Module	Responsibilities	Representative components
identity	Candidate resolution, alias matching, review	CandidateIdentityService, IdentifierRepository
mail	Gmail/Graph/forwarding, cursors	MailSyncService, MessageIngestionService
document	S3, scan, hash, PDF/OCR	DocumentStorageService, ResumeParseWorker
profile	Versions, skills, experience, promotion	LivingProfileService, PromotionPolicy
jobs	JD versions, requirements, search	JobService, JDAnalyzer
matching	Candidate retrieval, relevance, invalidation	MatchService, MatchWorker
hiring	Applications, submissions, stages, notes	ApplicationService, StageTransitionService
security	OIDC, tenant RBAC, presigned URLs	AuthorizationService, DocumentAccessService
ops	Outbox, retries, audit, monitoring	OutboxPublisher, AuditService

Use Java 21 or an approved supported JDK, Spring Boot, Spring Security, Spring Data JPA, Flyway, PostgreSQL, AWS SDK, SQS and S3. Start as a modular monolith with independently deployable worker processes; split microservices only after operational evidence justifies it.

D2. API contract inventory

Verb	Route	Purpose
POST	/api/v1/mail-connections	Create authorized source / setup
GET	/api/v1/mail-connections	List connections and health
DELETE	/api/v1/mail-connections/{id}	Revoke/disconnect
POST	/api/v1/uploads/presign	Authorized upload initiation
POST	/api/v1/uploads/complete	Confirm scanned processing
GET	/api/v1/candidates	Filter, sort, paginate; JD optional
GET	/api/v1/candidates/{id}	Current Living Profile
GET	/api/v1/candidates/{id}/versions	Historical versions
GET	/api/v1/candidates/{id}/applications	Application history
GET	/api/v1/resume-versions/{id}/view-url	Authorize signed PDF URL
POST	/api/v1/candidates/{id}/identity-review	Resolve match ambiguity
POST	/api/v1/jobs	Create job
POST	/api/v1/jobs/{id}/versions	Save new JD version
GET	/api/v1/jobs/{id}/matches	Paginated matching results
GET	/api/v1/applications	Job pipeline
PATCH	/api/v1/applications/{id}/stage	Transition with version check
GET	/api/v1/review-cases	Human review queue
GET	/api/v1/operations/health	Admin processing health
POST	/api/v1/data-requests	Export/deletion request

• All APIs authenticate, derive tenant from authorized membership, validate input, paginate lists and use stable error codes.

• Response example: candidate id, currentProfileVersion, currentResumeVersion, experienceMonths, skills, lastApplicationAt, sourceTypes, flags.

• Stage updates use If-Match/expectedVersion to return HTTP 409 on concurrent edits.

• Never expose raw S3 keys or accept tenant IDs as authorization proof.

• Use 202 Accepted + processing status for long-running uploads/JD extraction, not synchronous waiting.

D3. React architecture and performance

• React + TypeScript + Vite, accessible UI library, Tailwind tokens off-white #FCFBF7 and lavender #CBC3E3; keep color contrast sufficient.

• TanStack Query caches tenant-scoped API responses; debounce keyword searches and cancel stale requests.

• Candidate list uses server-side pagination (default 25), indexed sorting, skeletons and meaningful empty/error states.

• Never fetch resume PDF with list API. Viewer requests signed URL only after click; show metadata first and lazy-load PDF.js.

• Sidebar: Dashboard, Candidates, Jobs, Review Queue, Inbox Health, Settings. No unnecessary analytics on initial load.

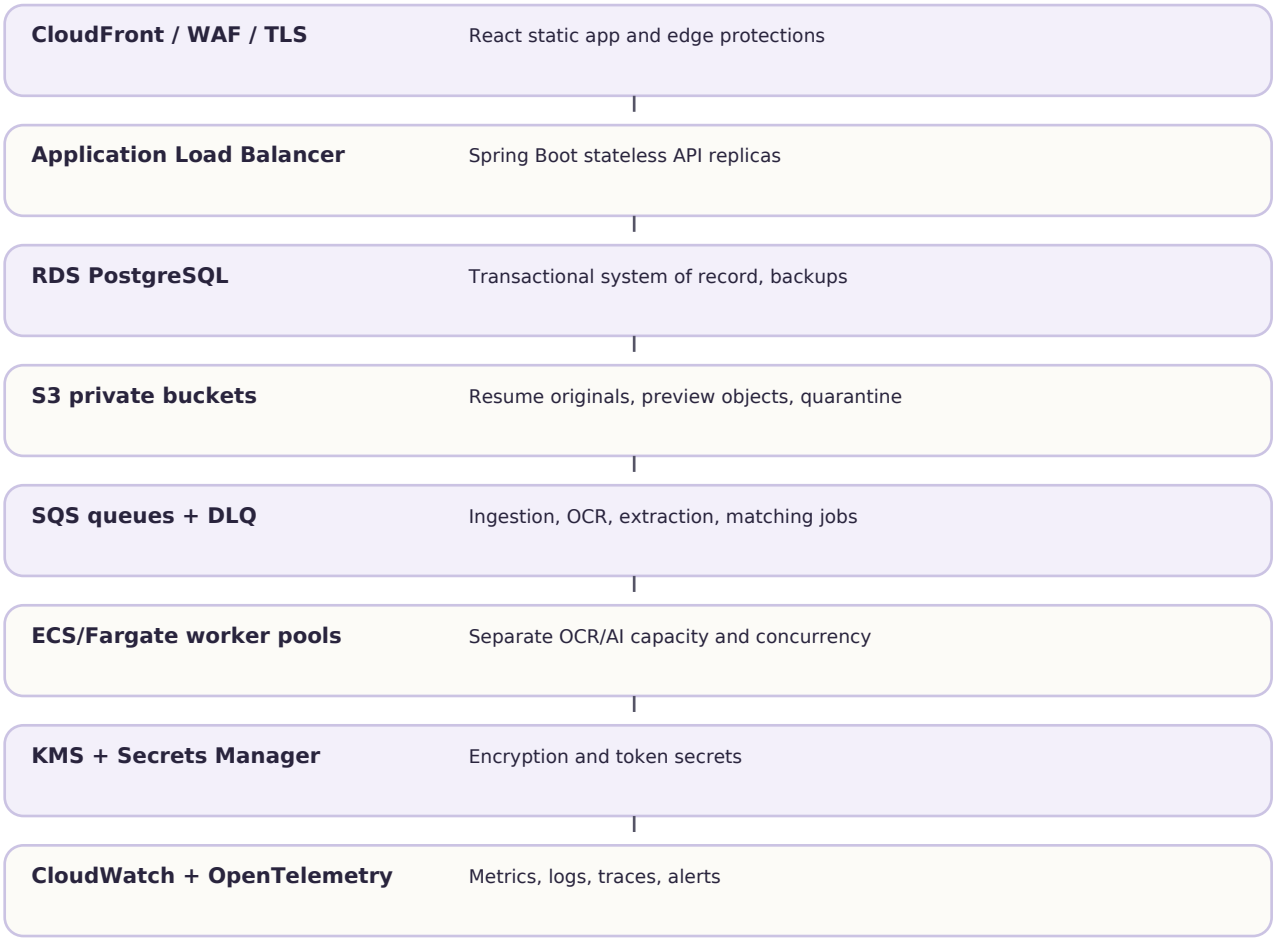
• Persist URL query parameters for filters and selected JD; allow back/forward navigation and shareable permitted views.

• Show source (Direct, Forwarded, Agency), latest resume badge, reapplication count, extraction confidence and last verified date.

• Separate admin operations dashboard from everyday HR workspace.

D4. Deployment, reliability and observability

AWS deployment reference



• Local development: Docker Compose PostgreSQL, RabbitMQ or LocalStack SQS, MinIO/S3-compatible storage, backend and React. Production adapters must preserve queue semantics.

• Use transactional outbox and idempotent consumers; no long database transaction across OCR/AI/network calls.

• Track queue oldest-message age, per-stage p95, sync cursor lag, dead letters, DB pool saturation, error budgets and AI spend.

• Daily backups and regular restore drills; schema migration review; staged rollout and rollback procedures.

E1. Complete bottleneck and failure-mode register

Risk / bottleneck	Mitigation
Mailbox throttling	Incremental cursors, backoff, reconciliation
Webhook notification loss	Periodic reconciliation; durable checkpoints
OAuth expiration/revocation	Health status, alerts, reconnect
Forwarded sender confusion	Separate forwarder provenance from candidate identity
Multiple attachments	Per-attachment records and candidate resolution
Duplicate provider deliveries	Unique message ID per connection
Repeated identical PDFs	Tenant SHA-256 dedup and extraction reuse
Same-day repeated applications	Separate submission events from application row
One-year newer resume	Versioned Living Profile and validated evidence
Old resume forwarded later	Document freshness validation, no receipt-only promotion
Out-of-order OCR completion	Guarded promotion / optimistic locking
Scanned image parsing	Bounded OCR fallback and review
Oversized PDFs/images	Page/size/memory limits, isolated workers
Malicious documents	Type sniffing, malware scan, sandbox

Risk / bottleneck	Mitigation
Resume prompt injection	Treat document as untrusted data, strict output schema
LLM hallucination	Schema validation, evidence, HR corrections
AI provider rate limits	Global/per-tenant limiter, backoff
AI cost explosion	Parse once, budgets, no bulk LLM matching
Queue overload	Separate queues, autoscaling, DLQ
Poison message	Bounded retries and terminal review
Transaction partial commit	Outbox and idempotent jobs
Slow database filters	Normalized skills, indexes, EXPLAIN
Hibernate N+1	DTO projections, fetch plans
DB connection exhaustion	Bounded Hikari pool, monitoring
Expensive dashboards	Aggregates, caching, 25-row page
Huge candidate table in browser	Server pagination, optional virtualization
Slow resume viewer	On-demand signed URL and PDF.js

Risk / bottleneck	Mitigation
Signed URL misuse	Tenant auth, short TTL, audit

Risk / bottleneck	Mitigation
Cross-tenant access	Tenant keys, RBAC, RLS option, tests
Cache data leakage	Tenant+permission-scoped keys
Concurrent HR stage edits	Optimistic version, history, 409
Stale JD scores	Versioned match keys and invalidation
Poor semantic ranking	Explainable weights, evaluation dataset
Candidate false merges	Conservative identity matching + review
Missing candidate email	Other signals, manual review
Duplicate skill synonyms	Canonical skills + alias table
Large Excel exports	Async streaming export
Storage cost growth	Retention and lifecycle controls
AI logs exposing PII	Redaction, restricted logs
Service outage	Graceful queueing, retry and alerts
No monitoring	SLOs, traces, ops dashboard
Deletion / data rights	Export, deletion workflow, holds

Risk / bottleneck	Mitigation
ATS adoption friction	CSV export, integrations later
Customer trust	No broad mailbox OAuth default, proof-based security
Weak market differentiation	Narrow inbox automation + Living Profile
Unfair auto-rejection	Human review and bias evaluation

E2. Security threat model and privacy controls

Threat	Impact	Control / test
Unauthorized mailbox access	Confidential email exposure	Dedicated mailbox/forwarding, least privilege, permission review
Cross-tenant candidate read	Privacy breach	Tenant scoped queries, FK, authorization and penetration tests
Leaked signed resume URL	PII disclosure	Short TTL, private bucket, auth before issuance, audit
Malicious PDF parser exploit	Remote execution risk	Sandbox, scanner, patched libraries, limits
Prompt injection via resume	Manipulated extraction or tools	Untrusted text boundary, no tool authority, strict schema
Token/secret compromise	Mail and storage access	KMS, secret refs, rotation, restricted IAM
Unwanted AI data reuse	Compliance and trust	Vendor contracts, minimize input, disable training where available
Retention without consent	Regulatory and reputation risk	Policy-based deletion, audit, legal review

E3. Test cases and measurable acceptance criteria

Test	Expected result
Same Gmail message delivered twice	One inbound message and one processing chain
One forwarded email with 10 resumes	10 attachment records and 10 independent candidate checks
Same PDF renamed	Same content hash, one physical object per tenant
V2 OCR completes before V1	V2 remains current; V1 never overwrites
Old agency resume received after new one	Do not downgrade profile
Candidate changes email	Safe link via evidence or review, no false merge
Same job repeated in one day	One active display row; all distinct submission events retained
Candidate reapplies next year	New attempt, new supported experience, old decision preserved
LLM unavailable	Job remains queued/reviewable; HR API still works
Two recruiters update same application	Conflict returned, no silent overwrite
Tenant A requests tenant B resume	Denied and audited
Large scanned PDF	Bounded resources and graceful review status
100k candidates indexed filters	Measure p95 latency against target
Disconnect mailbox	Sync stops; tokens revoked/deleted per policy

Suggested pilot targets (to validate, not promises): candidate search p95 <500 ms for common indexed filters; dashboard API p95 <700 ms; candidate detail p95 <300 ms; 99%+ eligible application capture in controlled tests; 95%+ agreed critical-field extraction accuracy; 50% administrative time reduction. Test against synthetic 100k candidate dataset and burst workloads.

E4. Implementation roadmap and deliverables

Phase	Work package	Exit criteria
0 Discovery	10-15 HR interviews, workflow audit, paid-pilot hypothesis	Clear target segment and success metrics
1 Foundation	Tenant auth, Flyway, S3, outbox, queues, observability	Tenant isolation and retry tests
2 Intake	Forwarding/dedicated inbox, sync, attachments, malware scan	No lost eligible messages in tests
3 Documents	PDF/DOCX/image OCR, extraction, evidence validation	Accuracy measured on representative sample
4 Living Profile	Identifiers, dedup, resume versions, snapshots, review	All repeat/forwarding edge cases pass
5 JD Search	JD versioning, normalized skills, indexed match	No full reparse; fast filtered results
6 Minimal UI	Dashboard, viewer, candidate detail, pipeline, privacy	HR can complete core flow unaided
7 Hardening	Load/security testing, restore drill, AI budgets	Pilot release criteria met
8 Pilot	Customer approved real-world deployment	Retention, time savings, willingness to pay measured

E5. Improvements I recommend beyond our agreed scope

• Add a field-level evidence viewer: clicking “6 years” shows the resume text/date that supports it. This is a trust and correction feature.

• Introduce a data-quality score and review SLA rather than forcing false certainty from AI.

• Support privacy-preserving forwarding as default; make broad OAuth opt-in and clearly disclosed.

• Offer ATS export/API integration before building a full ATS suite.

• Use job-specific evaluation datasets to calibrate matching weights and detect unfair exclusion.

• Add a candidate correction mechanism and track HR edits separately from AI-generated fields.

• Use tenant-level budgets for AI/OCR and cost dashboards before large-volume launch.

• Add controlled duplicate merge/unmerge operations with complete audit history.

• Introduce retention rules per tenant and source, plus legal holds and deletion reconciliation.

• Keep technical skill normalization catalog versioned; aliases evolve over time.

• Use a confidence threshold to distinguish auto-link, suggest match and human review; calibrate on real data.

• Provide a simple import wizard for historical resumes from ZIP/CSV/drive exports with provenance preserved.

• Instrument end-to-end time from email received to searchable profile, not only API latency.

• Make integration tests simulate provider replay, partial failures, retries, throttling and race conditions.

E6. Open decisions requiring product owner confirmation

Decision	Recommended default
What counts as a new application attempt after same-job repeat?	Same active application row; distinct submission events; new attempt when prior is closed or explicit policy
Should resume files be deleted when superseded?	Keep while required by applications, retention or holds; lifecycle cleanup later
Which cloud region/data residency?	Select with pilot customer and legal needs
Gmail/Outlook OAuth in MVP?	Forwarding + dedicated inbox first; OAuth after compliance review
What is the canonical experience policy?	Supported resume claim/employment dates; never add years based solely on elapsed time
Candidate identity merge threshold?	Conservative matching; conflicts go to review
How should unknown job applications be classified?	Unassigned inbox queue; allow HR to map to a JD
Should candidate notification emails be sent?	Not in initial MVP unless approved; consent and deliverability required
Pricing and tenant quotas?	Pilot validation before committing to tiers
Retention default and deletion obligations?	Legal and customer policy review before production

E7. Developer implementation checklist

- Create Flyway migration files for every table above with composite tenant foreign keys, checks and indexes.
- Define immutable inbound message and submission event contracts; write idempotency tests.
- Implement SHA-256 storage dedup transactionally and secure S3 object lifecycle.
- Implement candidate identity matching policy with explicit confidence thresholds and review state.
- Implement profile version promotion guarded by candidate version and evidence-based freshness.
- Implement structured AI extraction schemas and field provenance storage.
- Implement JD versioning and match cache invalidation via outbox.
- Implement permission-scoped React list queries, candidate profile and lazy resume viewer.
- Implement role/tenant integration tests, malicious file tests and race-condition tests.
- Instrument sync lag, queue age, throughput, p95 latency, AI spend and extraction error rates.
- Run customer discovery and pilot before broad feature expansion.

Appendix. Glossary and reference decisions

Term	Meaning
Candidate	Unique person identity within one organization
Resume file	One stored physical byte-identical document per tenant
Resume version	Candidate-associated document evidence with processing state
Current resume	Newest validated and approved version used for present matching
Profile version	Immutable structured snapshot of candidate information
Application	Logical job application/attempt and hiring stage
Submission	Distinct incoming event, including repeated same-day email
Inbound message	Source email envelope and provenance
Outbox	Transactional record to reliably publish asynchronous work
JD version	Immutable parsed job description and requirements
Match	Version-keyed relevance result with explanation
Review case	Human action needed for uncertain identity/data

End of consolidated source of truth. This document includes confirmed requirements from our conversation plus clearly labeled recommendations. Before development, approve open decisions, finalize migrations and perform privacy/legal review for intended customer markets.

VISUAL UI DESIGN ADDENDUM

Kartgen HireFlow AI | Master Blueprint V3

Dashboard wireframe

Search, Java/React/full-stack filters, candidate results

Candidate detail & resume viewer

On-demand PDF preview, candidate details and actions

Onboarding, jobs and pipeline

Privacy-first email setup and hiring workflow

Frontend components

Screen inventory, state and performance rules

Visual dashboard reference

Layout, sidebar, resume preview and color system

Original dashboard concept

Your initial wireframe and design direction

The complete 43-page engineering specification precedes this UI appendix.
Existing database, backend, risk and implementation sections remain intact.

Minimal dashboard UI wireframe

Off-white #FAF9F6, lavender #CBC3E3, calm typography, no oversized charts.

HireFlow AI

- **Dashboard**
- Candidates
- Jobs
- Pipeline
- Settings

Search name, email, skill or job...

Recruitment dashboard

MAIL CONNECTED

Total resumes	New applications	Needs review
1,240	**38**	**12**

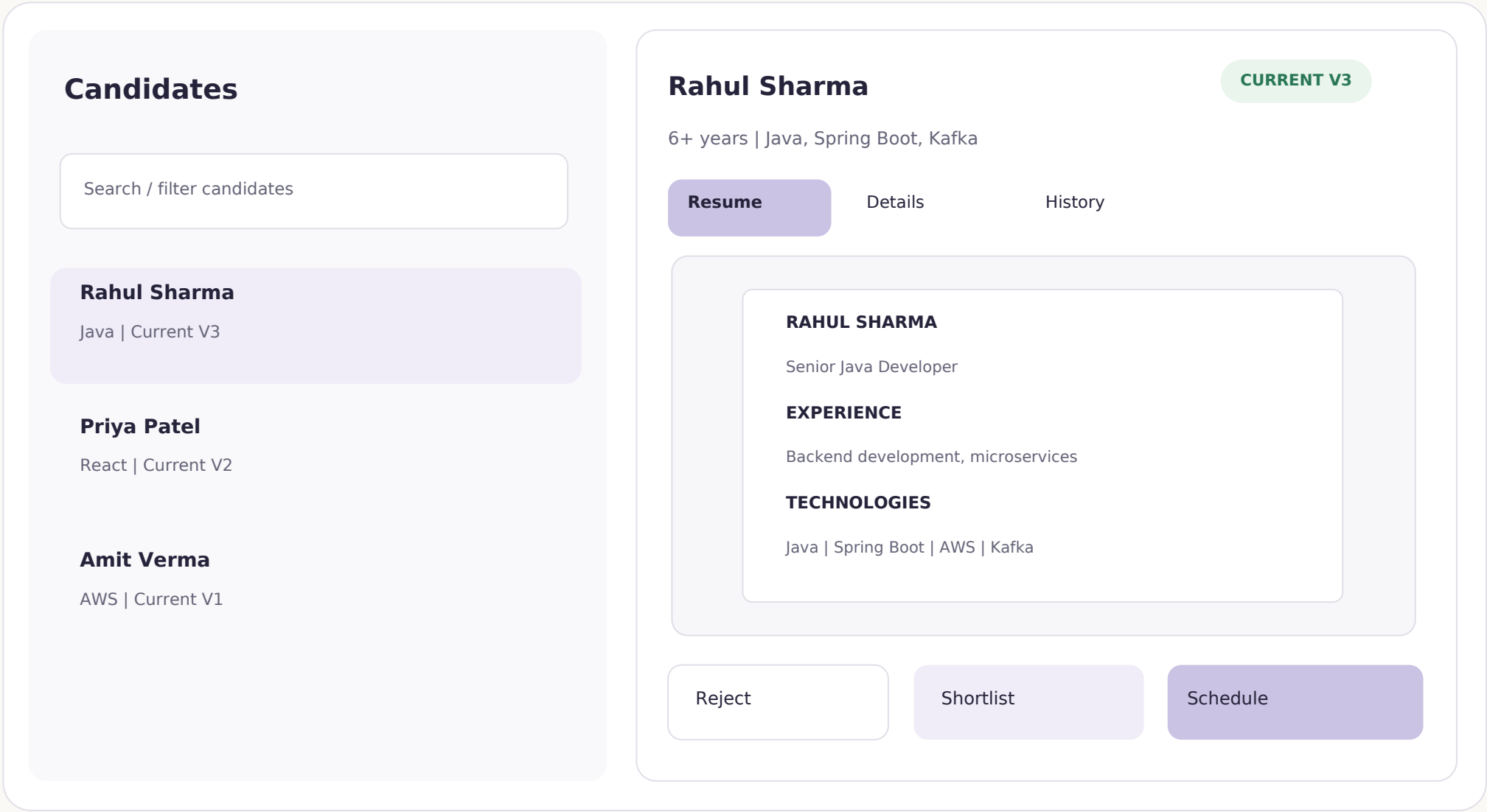
All resumes | Java React Python + Skills | Select JD

Candidate	Experience	Skills	Match	Resume
Rahul Sharma	6+ years	Java, Kafka	91%	View
Priya Patel	4 years	React, TS	--	View
Amit Verma	5 years	AWS, Docker	82%	View

View PDF only when clicked

Candidate detail + resume viewer wireframe

Open a drawer from results; fetch current validated resume only when requested.



Privacy-first onboarding and Jobs/Pipeline

Only five main nav items; email setup is a guided configuration, not a permanent dashboard burden.

Connect recruitment email

A. Dedicated inbox

careers@company.com

B. Controlled forwarding

No source-mailbox OAuth

C. Existing account OAuth

Broader read permission disclosure

Continue securely

Jobs

Java Backend Developer	82 matches
React Developer	42 matches

Hiring pipeline

New	Shortlisted	Interview
23	8	3

UI principle

Needs Review, candidate search and View Resume stay prominent. Advanced technical configuration lives in Settings.

React UI: screens, components and loading rules

Recommended implementation: Vite + TypeScript + Tailwind + shadcn/ui + TanStack Query/Table.

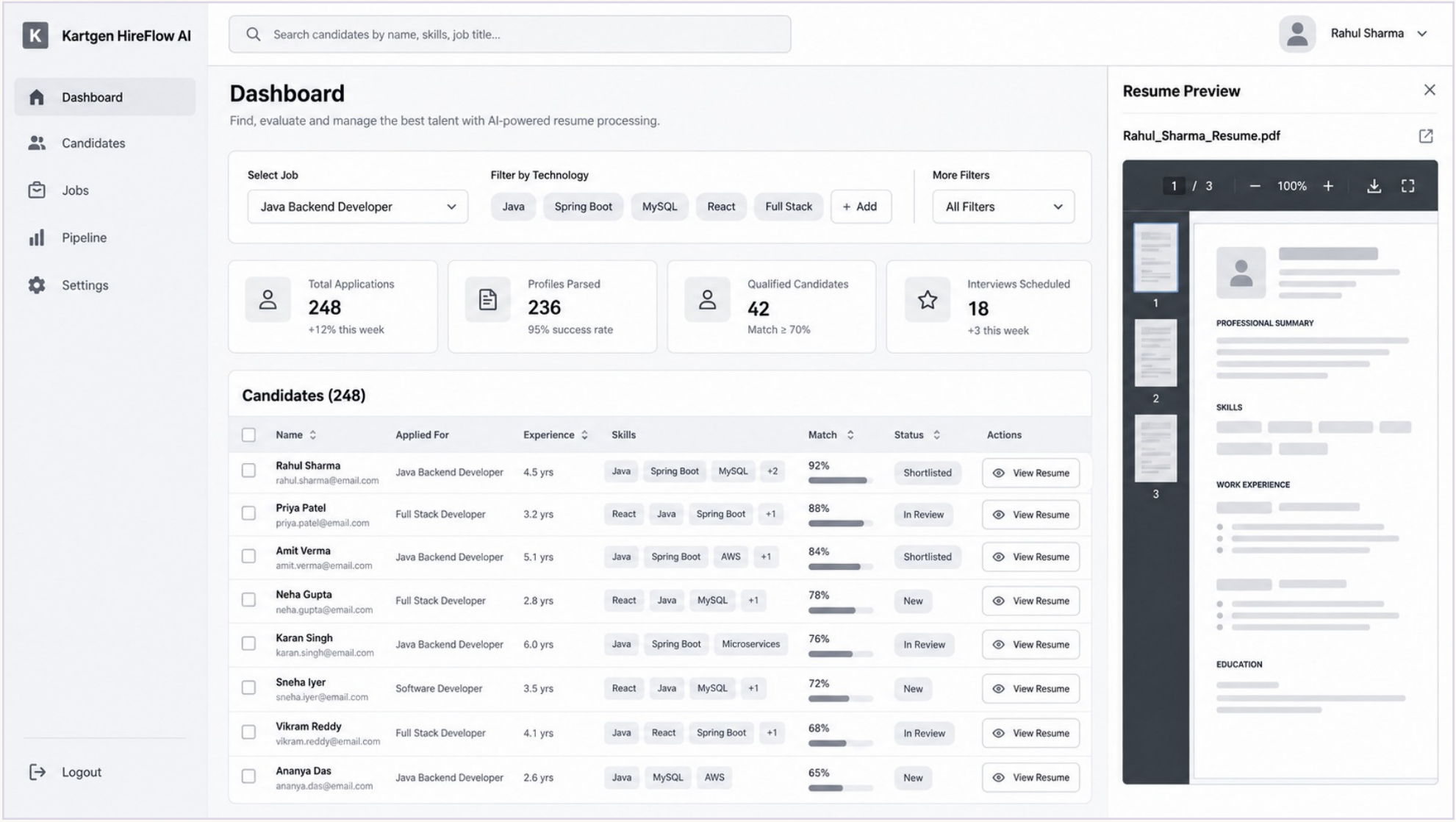
Screen / component	API data	Performance behavior
Login / onboarding	User, organization, connection status	No candidate data until authenticated.
Dashboard	Three KPIs, 10 recent applications, sync health	Parallel lightweight queries; no PDFs.
Candidates	Paginated candidate or application rows	Server filters, debounced text search.
Saved JD filter	Jobs and versioned match summaries	Reuse structured data; allow async match status.
Resume drawer	Candidate detail, one short-lived URL	Mount PDF.js only on request.
Jobs / pipeline	Open jobs, applications by stage	Invalidate affected query keys on updates.
Settings	Mail connection, roles, audit, retention	Expose access boundaries and sync failures.

- **Visual tokens**

  Background #FAF9F6 | Accent #CBC3E3 | Surface #FFFFFF | Text #25233A | thin borders / readable table.

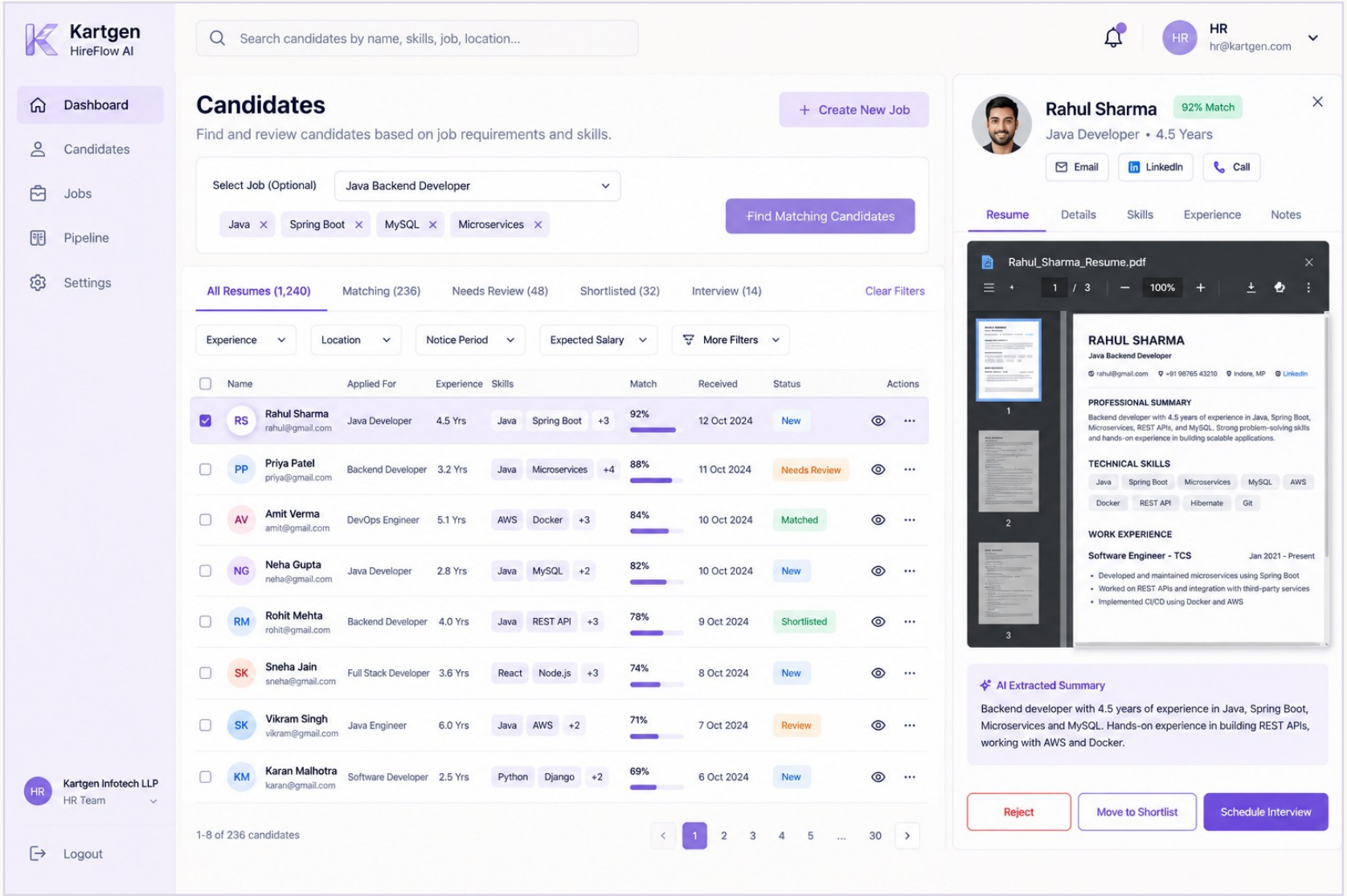
Minimal dashboard and resume preview

Off-white and lavender: candidate search, filters, table, and right-hand PDF preview.



Candidate list with document drawer

Alternate exploratory mockup: useful for reviewing document and candidate actions.



Your first dashboard wireframe

Original layout vision: sidebar, search, technology filters, and a large candidate results area.

